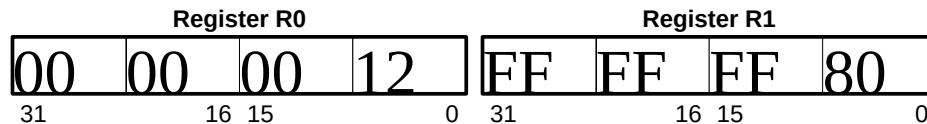


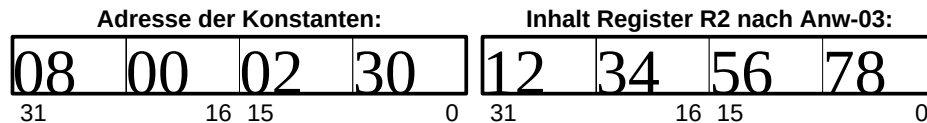
1 Laden von Konstanten in Register

Anw-01 Kleine Konstanten können direkt im Maschinenwort untergebracht werden.

Anw-02 Registerinhalte (im Hex-Format) nach Ausführung von Anw-02:



Anw-03 Größere Konstante erfordern einen zusätzlichen Speicherplatz zum Speichern des Wertes. Dieser kann automatisch vom Assembler erzeugt werden (Gleichheitszeichen). Durch Auswertung des Disassemblers kann die Adresse des Speicherplatzes bestimmt werden:



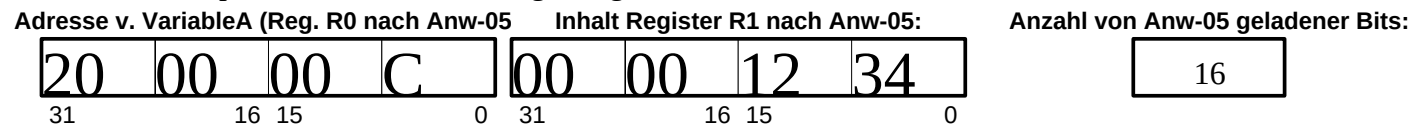
Zum Zugriff auf den zusätzlichen Speicherplatz setzt der Assembler eine besondere Assembleranweisung ein. Geben Sie an, welche Anweisung die CPU tatsächlich ausführt:

LDR R2, [PC, #56]

2 Zugriff auf Variable

Anw-04 Variable werden im Datenbereich des Prozessors untergebracht. Auf die Variablen kann nicht direkt zugegriffen werden, sondern es muss

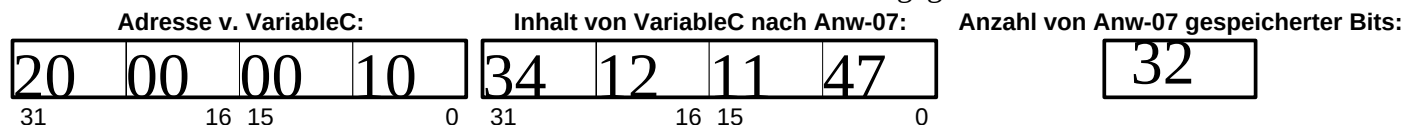
Anw-05 zuvor deren Speicheradresse in ein Register geladen werden:



Anw-06 Der Assembler überprüft nicht, ob der Lade/Speicherbefehl mit der Definition der Variable übereinstimmt. Anw-06 lädt 32 Bits, obwohl VariableA nur 16 Bit umfasst:



Anw-07 Die Adresse einer Variable kann auch als Differenz zu einer Basis angegeben werden:



3 Zugriff auf Felder

Die Elemente eines Feldes sind alle vom gleichen Typ und werden der Reihe nach im Speicher angelegt.

	Anfangsadresse:	Anzahl Elemente:	Anzahl Bytes pro Element:										
MeinHalbwortFeld:	<table><tr><td>20</td><td>00</td><td>00</td><td>14</td></tr><tr><td>31</td><td>16</td><td>15</td><td>0</td></tr></table>	20	00	00	14	31	16	15	0	<table><tr><td>6</td></tr></table>	6	<table><tr><td>2</td></tr></table>	2
20	00	00	14										
31	16	15	0										
6													
2													
MeinWortFeld:	<table><tr><td>20</td><td>00</td><td>00</td><td>20</td></tr><tr><td>31</td><td>16</td><td>15</td><td>0</td></tr></table>	20	00	00	20	31	16	15	0	<table><tr><td>6</td></tr></table>	6	<table><tr><td>4</td></tr></table>	4
20	00	00	20										
31	16	15	0										
6													
4													
MeinTextFeld:	<table><tr><td>20</td><td>00</td><td>00</td><td>38</td></tr><tr><td>31</td><td>16</td><td>15</td><td>0</td></tr></table>	20	00	00	38	31	16	15	0	<table><tr><td>9</td></tr></table>	9	<table><tr><td>1</td></tr></table>	1
20	00	00	38										
31	16	15	0										
9													
1													

Anw-08 Der Zugriff auf ein bestimmtes Element kann durch das Laden der Anfangsadresse in ein Register und die Angabe der Adressdifferenz für
Anw-09 das jeweilige Element erfolgen.

Anw-10

Register R1 nach Anw-09:

000022

3116150

Register R2 nach Anw-10:

00003E

3116150

Anw-11 Die Adressdifferenz kann auch über ein Register angegeben werden. Sie kann somit während des Programmlaufs verändert werden:
Anw-12

Register R4 nach Anw-12:				von Anw-12 gelesenen Elements:						
<table border="1"><tr><td>00</td><td>00</td><td>00</td><td>45</td></tr></table>				00	00	00	45	<table border="1"><tr><td>5</td></tr></table>		5
00	00	00	45							
5										
31	16	15	0							

Anw-13 Die Basisadresse kann automatisch nach Zugriff auf ein Element erhöht werden.

Register R0 nach Anw-13				Register R5 nach Anw-13:				Nr. (beginnend mit 0) des von Anw-13 gelesenen Elements:																		
<table><tr><td>20</td><td>00</td><td>00</td><td>16</td></tr><tr><td>31</td><td>16</td><td>15</td><td>0</td></tr></table>				20	00	00	16	31	16	15	0	<table><tr><td>00</td><td>00</td><td>00</td><td>3e</td></tr><tr><td>31</td><td>16</td><td>15</td><td>0</td></tr></table>				00	00	00	3e	31	16	15	0	<table><tr><td>1</td></tr></table>		1
20	00	00	16																							
31	16	15	0																							
00	00	00	3e																							
31	16	15	0																							
1																										

Anw-14

Register R0 nach Anw-14

200018

3116150

Register R6 nach Anw-14:

0000FFCC

3116150

Nr. (beginnend mit 0) des von Anw-14 gelesenen Elements:

2

Anw-15

Register R0 nach Anw-15

Adresse des von Anw-15 überschriebenen Elements:

Nr. (beginnend mit 0) des von Anw-15 überschriebenen Elements:

20	00	00	1A	20	00	00	1A	
31	16	15	0	31	16	15	0	

3

4 Addition und Subtraktion von vorzeichenlosen und vorzeichenbehafteten Integer-Werten

Arithmetik-Befehle haben 3 Operanden. Tragen Sie in das nachfolgende Schema die beteiligten Werte in binärer Form ein. Wandeln Sie die binäre Darstellungen um in vorzeichenbehaftete und vorzeichenlose Dezimalzahlen. Überprüfen Sie die Rechnung. Stimmen die Ergebnisse mit den Erwartungen überein?

Anw-16
Anw-17
Anw-18
Anw-19

		dezimale Interpretation der Bitmuster	
		signed	unsigned
Reg. R1:	00010010001101000101011001111000	305419896	305419896
Reg. R2:	100111011110010100101100110000110	-1647683194	2647284102
Reg. R3:	10101111111111101010111111111110	-1342263298	2952703998
31 16 15 0			
Kreuzen Sie die gesetzten Bits des Statusregisters an: V C N Z 		Ergebnis: Ok Nicht Ok	Ergebnis: Ok Nicht Ok

Anw-20
Anw-21
Anw-22

		dezimale Interpretation der Bitmuster	
		signed	unsigned
Reg. R4:	11001100000000000000000000000000	-872415232	3422552064
Reg. R5:	01001110000000000000000000000000	1308622848	1308622848
Reg. R6:	00011010000000000000000000000000	436207616	436207616
31 16 15 0			
Kreuzen Sie die gesetzten Bits des Statusregisters an: V C N Z 		Ergebnis: Ok Nicht Ok	Ergebnis: Ok Nicht Ok

Anw-23
Anw-24
Anw-25

		dezimale Interpretation der Bitmuster	
		signed	unsigned
Reg. R7:	00100111000000000000000000000000	654311424	654311424
Reg. R8:	01000101000000000000000000000000	1157627904	1157627904
Reg. R9:	11100010000000000000000000000000	-503316480	3791650816
31 16 15 0			
Kreuzen Sie die gesetzten Bits des Statusregisters an: V C N Z 		Ergebnis: Ok Nicht Ok	Ergebnis: Ok Nicht Ok